

Universidad San Carlos de Guatemala

Organización de Lenguajes y Compiladores 2

Ing. Mario Tobar



MANUAL TÉCNICO

Mario Raúl Yancor Ocaña

201930761

Quetzaltenango, 26 de septiembre de 2026

ÍNDICE

Presentación y alcance	3
Tecnologías utilizadas	3
Arquitectura y estructura del proyecto	4
Flujo interno del compilador	5
Análisis léxico, sintáctico y construcción del AST	6
Construcción del AST	6
Especificación de palabras reservadas, símbolos y gramática	7
Análisis semántico e importaciones	10
Modelo de memoria: Stack y Heap	11
Bajado de cuartetas a memoria	12
Ejemplo de transformación	13
Generación de C3D	13
Generación de código C	13
Coloreado en tiempo real	14
Interfaz y gestión del árbol de trabajo	14
Manejo de errores y validación del proyecto	15
Tabla de compatibilidad de tipos para los tres lenguajes	15
Operadores lógicos y relacionales	17
Pruebas y validación	17
Clases más importantes y complejas	18
Diagrama de clases	18
Compilación y ejecución	20

Presentación y alcance

Contacto 3xtrat3rr3str3D es una aplicación de escritorio desarrollada en Java para analizar y compilar un proyecto formado por tres lenguajes: Y? (.y), Zetariano (.z) y Pig Latin (.pig). El archivo Pig Latin funciona como punto de entrada del proyecto; desde él se importan funciones y estructuras de Y?, y clases de Zetariano.

La aplicación ejecuta análisis léxico, sintáctico y semántico antes de iniciar la generación. Cuando el proyecto es válido, construye AST propios, resuelve importaciones y tipos, genera cuartetos, planifica memoria, baja las operaciones a Stack/Heap, produce código de tres direcciones y finalmente genera un archivo C compilable.

Objetivos técnicos principales:

- Reconocer los tres lenguajes mediante gramáticas ANTLR4 independientes.
- Detener la compilación si cualquier archivo presenta un error léxico, sintáctico o semántico.
- Representar el programa mediante AST propios y una tabla de símbolos común.
- Generar cuartetos independientes del lenguaje de origen.
- Aplicar un modelo explícito de Stack y Heap para variables, parámetros, arreglos, estructuras, objetos y cadenas.
- Generar C3D y código C ejecutable a partir de las cuartetos bajadas.
- Permitir abrir, guardar y descargar archivos o carpetas desde la interfaz.
- Colorear Y?, Zetariano y Pig Latin en tiempo real sin una biblioteca externa de resaltado.

Tecnologías utilizadas

Tecnología	Versión	Uso
Java	release 21	Lenguaje del proyecto. El pom.xml compila con Java 21; el desarrollo puede ejecutarse sobre un JDK más reciente.
Apache Maven	3.9.x	Gestión del proyecto, dependencias, compilación, generación ANTLR y ejecución de pruebas.
ANTLR4	4.13.2	Generación de los lexers y parsers de Y?, Zetariano y Pig Latin.
JUnit Jupiter	5.11.4	Pruebas unitarias, semánticas, de regresión y de integración.
Java Swing	Java SE	Interfaz gráfica, editores, tablas, pestañas y árbol del proyecto.
Java NIO	Files / Path	Apertura, guardado, copia y empaquetado de archivos/carpetas.
GCC / MinGW	C	Validación externa del código C generado: compilación y ejecución del binario.
Git / GitHub	Control de versiones	Repositorio, ramas y respaldo del proyecto.
IntelliJ IDEA	IDE	Edición, ejecución de Maven y depuración.

Configuración principal del pom.xml

```
<properties>
  <maven.compiler.release>21</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <antlr.version>4.13.2</antlr.version>
  <junit.version>5.11.4</junit.version>
</properties>
```

Arquitectura y estructura del proyecto

La solución usa una arquitectura por fases. Cada paquete concentra una responsabilidad y la clase `CompiladorLatinus` actúa como fachada: recibe texto, ejecuta análisis, decide si puede publicar artefactos y devuelve un `ResultadoCompilacion` inmutable.

Paquete	Clases representativas	Responsabilidad
antlr	YLexer/YParser, ZetarianoLexer/ZetarianoParser, PigLatinLexer/PigLatinParser	Reconocimiento léxico y sintáctico.
analisis	AnalizadorArchivo, YAstBuilder, ZAstBuilder, PAstBuilder	Recolecta errores y construye AST propios.
ast	ProgramaAst, nodos comunes, PAst, ZAst	Modelo estructurado e independiente del parse tree.
semantica	AnalizadorSemanticoY/Z/Pig, TablaSimbolos, ResolutorImportsPig	Tipos, ámbitos, nombres, llamadas, imports y sobrecarga.
ir	Cuarteta, ProgramaIntermedio, GeneradorCuartetas*, GeneradorIntermedioProyecto	Representación intermedia común.
memoria	PlanificadorMemoriaProyecto, BajadorMemoria/Arreglos/Llamada s/Cadenas/Inicializadores	Asignación de Stack/Heap y traducción a operaciones de memoria.
c3d	GeneradorCodigoTresDirecciones, GeneradorCodigoC	Salida textual C3D y código C.
editor	EditorCodigo, ColoreadorAntlr, NumeradorLineas	Editor y coloreado en tiempo real.
interfaz	VentanaPrincipal, PanelCuartetas, PanelCodigoC3D, PanelCodigoC	Presentación de diagnósticos y resultados.

Paso	Resultado
1	Archivos .y / .z / .pig
2	Lexer + Parser ANTLR4
3	AST propio
4	Análisis semántico + imports + tabla de símbolos
5	Cuartetas / ProgramaIntermedio
6	Plan de memoria
7	Bajado Stack / Heap
8	C3D
9	Código C
10	GCC -> ejecutable

Flujo por capas

Regla	Producción
expresion	expresionOr
expresionOr	expresionAnd (expresionAnd)*
expresionAnd	expresionIgualdad (&& expresionIgualdad)*
expresionIgualdad	expresionRelacional ((== !=) expresionRelacional)*
expresionRelacional	expresionAditiva ((< > <= >=) expresionAditiva)*
expresionAditiva	expresionMultiplicativa ((+ -) expresionMultiplicativa)*
expresionMultiplicativa	expresionUnaria ((* /) expresionUnaria)*
expresionUnaria	(non - +) expresionUnaria expresionPrimaria
expresionPrimaria	literal llamada referencia inicializadorEstructura (expresion)
referencia	ID ([expresion] . ID)*
llamadaFuncion	ID (listaExpresiones?)

Flujo interno del compilador

1. VentanaPrincipal solicita a GestorProyecto los archivos reconocidos del árbol de trabajo.
2. CompiladorProyecto ejecuta AnalizadorArchivo sobre todos los .y, .z y .pig.

3. AnalizadorArchivo crea el lexer y parser correctos según LenguajeFuente y recolecta errores ANTLR.
4. Si un archivo tiene errores léxicos o sintácticos, no se construye su AST y el proyecto completo queda inválido.
5. Los AST válidos pasan a AnalizadorSemanticoY, AnalizadorSemanticoZ y AnalizadorSemanticoPig.
6. ResolutorImportsPig vincula importaciones .y/.z con los símbolos definidos en esos archivos.
7. Si la semántica completa es válida y existe exactamente un .pig principal, GeneradorIntermedioProyecto produce las cuartetas.
8. PlanificadorMemoriaProyecto calcula marcos de Stack, layouts de Heap, campos, referencias y tamaños.
9. Los bajadores convierten construcciones de alto nivel en LEER/ESCRIBIR_STACK, LEER/ESCRIBIR_HEAP, saltos, temporales y llamadas.
10. GeneradorCodigoTresDirecciones muestra el C3D y GeneradorCodigoC produce el archivo C final.

Análisis léxico, sintáctico y construcción del AST

Análisis léxico

Cada lenguaje posee su propio lexer. Las palabras reservadas aparecen antes de IDENTIFICADOR, por lo que un lexema reservado no puede ser utilizado accidentalmente como nombre de variable. Los tokens conservan línea y columna para construir diagnósticos.

Y? necesita un paso adicional: TokenizadorIndentacionY transforma la indentación significativa en tokens INDENT y DEDENT. Por eso los bloques de Y? se reconocen con la misma idea de Python, sin llaves.

Análisis sintáctico

Los parsers ANTLR validan la forma del programa. Y? exige %funciones y permite %estructuras; Zetariano exige una única clase pública por archivo; Pig Latin contiene imports opcionales, VARIABLES> opcional y MAIOR>...FINIS; obligatorio.

Construcción del AST

El proyecto no trabaja directamente con el parse tree después del análisis. YAstBuilder, ZAstBuilder y PASTBuilder convierten los contextos ANTLR en nodos del dominio. Esto simplifica la semántica y permite que la generación de cuartetas sea independiente de ANTLR.

AST	Ejemplos de nodos	Uso
Común / Y?	ProgramaAst, FuncionAst, SiAst, ParaAst, BinariaAst, AccesoArregloAst	Funciones, estructuras, expresiones y controles de Y?.
Zetariano	ZAst.Clase, Constructor, Metodo, Atributo, NuevaInstancia, Ternaria	Modelo orientado a objetos.
Pig Latin	PAst.Programa, Importacion, Declaracion, Entrada, Salida, NuevoObjeto	Punto de entrada e integración entre lenguajes.

Especificación de palabras reservadas, símbolos y gramática

Lenguaje Y? (.y)

Y? es case sensitive, usa indentación significativa, no permite variables globales y organiza el archivo en %estructuras (opcional) y %funciones (obligatorio). Las estructuras válidas se definen únicamente con la forma indentada.

Categoría	Palabras reservadas
Secciones	%estructuras, %funciones
Estructuras/funciones	estructura, definir, retornar
Condicionales	si, entonces, sino, contrario
Selección	elegir, caso, siempre
Ciclos	para, mientras, hacer
Control	romper, continuar
Entrada/salida	imprimir, leer
Tipos	entero, flotante, cadena, caracter, bool
Booleanos	verdadero, falso

Símbolos principales: + - * / == != < > <= >= && || ! ++ -- -> : ; , . () [] { }

Gramática condensada de Y?

archivo

```
: seccionEstructuras?
    seccionFunciones
EOF
```

estructura

```
: 'estructura' ID ':' NUEVA_LINEA
    INDENT atributo+ DEDENT
```

funcion

```
: 'definir' ID '(' parametros? ')' ('->' tipo)? ':'
    NUEVA_LINEA bloque
```

parametro

```
: tipo ID
| '[' tipo ID
| '{' tipo ID
```

si

```
: 'si' '(' expresion ')' 'entonces' bloque
    ('sino' '(' expresion ')' 'entonces' bloque)*
    ('contrario' bloque)?
```

elegir

```
: 'elegir' '(' expresion ')' ':'
    caso+ siempre?
```

para

: 'para' '(' inicio ';' condicion ';' actualizacion ')' ':' bloque

Los parámetros [] tipo nombre representan arreglos por referencia. Los parámetros {} Tipo nombre representan estructuras por referencia. Las dimensiones de los arreglos deben ser enteras y los arreglos se almacenan aplanados a bajo nivel.

Lenguaje Zetariano (.z)

Zetariano es un lenguaje orientado a objetos inspirado en Java. Cada archivo .z contiene una clase y el nombre del archivo debe coincidir con el nombre real de la clase obtenido desde el AST.

Categoría	Palabras reservadas
Clase/visibilidad	public, class
Tipos/retorno	void, int, double, char, boolean, String
Objetos	new, null
Retorno	return
Condicionales	if, else, switch, case, default
Ciclos	for, while, do
Control	break, continue
Entrada/salida	print, println, readln
Booleanos	true, false

Símbolos principales: + - * / % = += -= *= /= == != < > <= >= && || ! ++ -- ? : ; , . () [] { }

Gramática condensada de Zetariano

archivo

: clase EOF

clase

: 'public' 'class' ID '{' 'miembro* '}'

miembro

: atributo

| constructor

| metodo

constructor

: 'public' ID '(' parametros? ')' bloque

metodo

: 'public' (tipo | 'void') ID '(' parametros? ')' bloque

if

: 'if' '(' expresion ')' sentencia ('else' sentencia)?

switch

: 'switch' '(' expresion ')' '{'

('case' expresion ':' sentencia*

| 'default' ':' sentencia*)*

}

for

: 'for' '(' inicio? ';' condicion? ';' actualizacion? ')' sentencia

expresion

: asignacion

| ternaria

| logica

| relacional

| aritmetica

| llamada/acceso/new

Se soportan constructores y métodos sobrecargados. La resolución selecciona la mejor firma: coincidencia exacta primero, después promociones válidas como char -> int e int -> double. Los objetos viven en Heap.

Lenguaje Pig Latin (.pig)

Pig Latin es el lenguaje principal del proyecto. No define estructuras propias: debe importarlas desde Y?. Las clases se importan desde Zetariano. El compilador exige exactamente un archivo .pig por proyecto.

Categoría	Palabras reservadas / tokens
Secciones	VARIABLES>, MAIOR>, FINIS;
Importación	import
Declaraciones	esto, series
Objetos	novus
Condicional	si, aliter, finis
Ciclos	dum, facere, per
Control	perge, interrompe
Tipos	numerus, decimalis, textum, littera, bool
Booleanos	verum, falsus
Entrada/salida	<<, >>

Símbolos principales: + - * / = == != < > <= >= && || ! ++ -- : ; , . () [] { } <<>>

Gramática condensada de Pig Latin

archivo

: import*

variables?

principal

EOF

import

: 'import' ruta ('.y' | '.z') ';;'?

variables

: 'VARIABLES>' declaracion*

principal

: 'MAIOR>' sentencia* 'FINIS;'

declaracion

: 'esto' ID ':' definicion ':'

| 'series' ID dimension+ ':' tipo inicializador? ':'

entrada

: destino '<<' ':'?

| '<<' ':'?

salida

: '>>' expresion ('>>' expresion)* ':'?

comentarioEspecial

: '##' contenido '##'

La lectura con destino se escribe variable <<. La forma << sin destino consume una línea y la descarta. La forma << variable no pertenece a la gramática final. Los bloques ## ... ## son comentarios multilínea.

Análisis semántico e importaciones

Después de construir los AST, cada lenguaje pasa por un analizador semántico específico. La semántica comprueba declaraciones, ámbitos, tipos, acceso a arreglos/campos, contexto de break/continue, retornos, llamadas, constructores y condiciones booleanas.

Componente	Responsabilidad
AnalizadorSemanticoY	Funciones, estructuras, arreglos, paso por valor/referencia, elegir/casos y tipos Y?
AnalizadorSemanticoZ	Clases, atributos, constructores, métodos, sobrecarga, new/null, ternario y tipos Z.
AnalizadorSemanticoPig	Variables globales, arreglos, objetos/estructuras importadas, lectura/salida y controles Pig.
ResolutorImportsPig	Conecta importaciones .y/.z con funciones, estructuras, clases y métodos disponibles.
TablaSimbolos / Ambito	Mantienen nombre, categoría, tipo declarado y resolución por alcance.

Política de validez global

Un archivo inválido no se elimina silenciosamente. CompiladorProyecto analiza todos los archivos y exige que todos sean válidos. Si existe cualquier error léxico, sintáctico o semántico, el resultado global es inválido y no se genera backend.

Importaciones Pig Latin

Los imports se resuelven por ruta y extensión. Un .y aporta funciones y estructuras; un .z aporta una clase. El semántico evita referencias ambiguas y comprueba que las llamadas/constructores coincidan con una firma compatible.

Generación de cuartetas y programa intermedio

Una cuarteta representa una instrucción con cuatro campos: operador, argumento 1, argumento 2 y resultado. Los generadores `GeneradorCuartetasY`, `GeneradorCuartetasZ` y `GeneradorCuartetasPig` recorren los AST y producen la misma representación común.

Operador	Ejemplo conceptual	Significado
ASIGNAR	ASIGNAR t0 - x	Guardar un valor.
SUMAR	SUMAR a b t0	Calcular a + b en temporal.
MENOR	MENOR i t0 t1	Comparación relacional.
ETIQUETA	ETIQUETA - - L0	Destino de salto.
SALTAR	SALTAR - - L0	Goto incondicional.
SALTAR_SI_FALSO	SALTAR_SI_FALSO t1 - L1	Control condicional.
PARAMETRO	PARAMETRO x - -	Preparar argumento de llamada.
LLAMAR	LLAMAR funcion n t2	Invocar función/método.
RETORNAR	RETORNAR x - -	Retorno de función.
IMPRIMIR / LEER	IMPRIMIR x println -	Entrada/salida.
NUEVO_ARREGLO	NUEVO_ARREGLO tipo dims ref	Reserva lógica de arreglo.
NUEVO_OBJETO	NUEVO_OBJETO Clase args ref	Reserva lógica de objeto.

`GeneradorIntermedioProyecto` une los programas de los tres lenguajes. Los temporales y etiquetas internas usan un espacio de nombres protegido y luego se prefijan por origen (`y_`, `z_`, `pig_`) para evitar colisiones con identificadores del usuario como `t0` o `L0`. Los literales de cadena no se modifican durante este renombrado.

Modelo de memoria: Stack y Heap

El código C generado implementa explícitamente dos áreas de memoria globales. Stack representa marcos de llamadas, variables locales, parámetros y valores de retorno. Heap representa datos dinámicos: objetos, estructuras, arreglos y cadenas.

Modelo base generado en C

```
#define STACK_SIZE 100000
#define HEAP_SIZE 1000000
```

```
double Stack[STACK_SIZE];
double Heap[HEAP_SIZE];
```

```
int P = 0;
int H = 0;
```

Elemento	Memoria	Representación
Variable primitiva local/global	Stack	Valor numérico, carácter o booleano codificado.
Parámetro de función	Stack	Valor o referencia según el lenguaje/tipo.
Retorno	Stack	Slot reservado en el marco.
Objeto Z	Heap	Bloque contiguo de atributos.
Estructura Y?	Heap	Campos en offsets calculados por <code>LayoutHeapProyecto</code> .

Arreglo	Heap	Datos aplanados; se conserva referencia base y dimensiones.
String/cadena/textum	Heap	Caracteres codificados y terminador -1.
null / referencia no inicializada	Stack o Heap	-1.
Primitivo no inicializado	Stack o Heap	0.

Stack

P apunta a la base del marco activo. PlanificadorMemoriaProyecto calcula offsets de parámetros y variables. Las funciones no usan variables C reales para representar el lenguaje fuente: los valores se escriben y leen desde Stack.

Accesos relativos al puntero P

lectura conceptual

```
mem_t0 = Stack[P + 2];
```

escritura conceptual

```
Stack[P + 3] = mem_t0;
```

Heap

H indica la próxima posición libre. Cuando se crea un objeto, estructura, arreglo o cadena, se reserva espacio a partir de H y la referencia resultante es la posición inicial. Los accesos a campos se transforman en referenciaBase + offset.

Los arreglos multidimensionales se aplanan. Un índice lógico se convierte en un desplazamiento lineal antes de leer o escribir Heap. Los arreglos Y? pasados como [] tipo se transmiten por referencia; por eso una modificación hecha dentro de una función es visible en el llamador.

Recursividad

La recursividad se soporta porque cada llamada mueve P a un nuevo marco. Al retornar, P vuelve al marco anterior. La prueba integral factorial(5) produjo 120 en el ejecutable C, validando el manejo de marcos recursivos.

Bajado de cuartetos a memoria

El programa intermedio contiene operaciones de alto nivel. Antes de generar C, una cadena de bajadores las convierte en operaciones explícitas de memoria.

Clase	Responsabilidad
BajadorInicializadoresProyecto	Expande inicializadores compuestos, estructuras anidadas y referencias existentes.
BajadorArreglosProyecto	Reserva arreglos, calcula posiciones y baja lecturas/escrituras a Heap.
BajadorLlamadasProyecto	Convierte llamadas, métodos y constructores en preparación de marcos, parámetros y retornos.
BajadorCadenasProyecto	Reserva literales en Heap, propaga referencias String y transforma concatenaciones.
BajadorMemoriaProyecto	Sustituye declaraciones/accesos por operaciones Stack/Heap usando el plan de memoria.

Ejemplo de transformación

Ejemplo conceptual

Cuarteta de alto nivel:

```
SUMAR nombre "!" t0
```

Después de bajar cadenas/memoria:

```
LEER_STACK P+2 String mem_t0
```

```
SUMAR @STR@mem_t0 @STR@str_t0 z_t0
```

```
ESCRIBIR_STACK z_t0 - P+3
```

El marcador interno de cadena permite que el generador sepa que un temporal representa una referencia a texto y no un número. Ese metadato se conserva aunque el String haya pasado por un parámetro, atributo o arreglo.

Generación de C3D

GeneradorCodigoTresDirecciones convierte las cuartetas ya bajadas en una representación textual legible. Esta vista sirve para explicar cómo se ejecutará el programa antes de traducirlo a C.

Ejemplo de C3D

```
func MAIOR
```

```
mem_t0 = Stack[P + 1]
```

```
t0 = mem_t0 + 1
```

```
Stack[P + 1] = t0
```

```
ifFalse t0 goto L1
```

```
print(t0)
```

```
goto L2
```

```
L1:
```

```
print(0)
```

```
L2:
```

```
endfunc
```

Los temporales almacenan resultados parciales; las etiquetas representan destinos de salto. Los ciclos y condicionales se reducen a comparaciones, saltos condicionales, saltos incondicionales y etiquetas.

Generación de código C

GeneradorCodigoC recibe el programa de memoria y emite C. El resultado contiene includes, Stack, Heap, punteros P/H, temporales globales o locales, funciones auxiliares de runtime, una función C por función/método del proyecto y main(void) como punto de entrada.

Característica	Implementación C
Impresión numérica	printf con formato numérico.
Impresión de String	runtime_print_string recorre Heap hasta -1.
Lectura numérica	runtime_read_number consume una línea y convierte.
Lectura String	runtime_read_string guarda caracteres en Heap.
Lectura char/bool	Funciones tipadas del runtime.
Concatenación	Funciones runtime que copian cadenas/números hacia Heap.
Objetos/estructuras	Direcciones base y offsets de Heap.
Arreglos	Índices linealizados y lecturas/escrituras Heap.

División entera Z	Se preserva truncamiento hacia cero.
char / boolean en Z	char imprime carácter; boolean imprime true/false.

El archivo generado puede compilarse con GCC. En las pruebas se usa habitualmente -lm para máxima compatibilidad con operaciones matemáticas; programas que no requieren funciones de libm también pueden compilar sin esa opción.

Compilación del archivo generado

```
gcc programa.c -o programa.exe -lm
./programa.exe
```

Coloreado en tiempo real

El coloreado se realiza con ColoredAntlr. No se utiliza una biblioteca externa de syntax highlighting. Se reutilizan los lexers ANTLR únicamente para identificar la categoría de cada token y aplicar atributos Swing.

Token	Estilo aplicado
SECCION_* / FIN_PROGRAMA	Azul y negrita.
KW_*	Morado y negrita.
TYPE_*	Verde azulado y negrita.
BOOL_* / NULL_VALUE	Naranja y negrita.
ENTERO / DECIMAL	Rojo.
CADENA / CHARACTER	Verde.
COMENTARIO_*	Gris y cursiva.
Resto	Color normal.

EditorCodigo usa un DocumentListener y un Timer de 150 ms. Cuando el usuario escribe, el temporizador se reinicia; al pasar 150 ms sin cambios se ejecuta colorearAhora(). Este pequeño retraso evita recolorear todo el documento por cada tecla.

Flujo del resaltado

```
DocumentListener -> cambioTexto()
|
v
Timer(150 ms)
|
v
colorearAhora()
|
v
lexer del lenguaje -> tokens -> estilos
```

Interfaz y gestión del árbol de trabajo

VentanaPrincipal coordina el árbol del proyecto, editores y pestañas de resultados. Los archivos se abren en EditorCodigo y el lenguaje se determina por extensión.

Pestaña/área	Contenido
Diagnósticos	Errores léxicos, sintácticos, semánticos y de proyecto.
Cuartetas	Operador, argumento 1, argumento 2 y resultado.
Memoria / C3D	Cuartetas después del bajado a Stack/Heap.
Código C3D	Representación textual de tres direcciones.
Código C	Programa C final listo para guardar/compilar.
Árbol	Carpetas y archivos del proyecto.
Editor	Código fuente con numeración de líneas y coloreado.

La gestión de archivos permite abrir un proyecto, guardar el archivo activo, guardar todos, descargar un archivo seleccionado y descargar una carpeta/proyecto como ZIP. Al comprimir se omiten directorios de trabajo como target, .git e .idea.

Manejo de errores y validación del proyecto

RecolectorErroresAntlr captura errores de lexer y parser con archivo, línea, columna, tipo y mensaje. Los analizadores semánticos agregan diagnósticos de tipos, símbolos, contexto e imports.

Tipo	Ejemplos
LÉXICO	Símbolo no reconocido, literal mal formado.
SINTÁCTICO	Token faltante, bloque incompleto, orden inválido.
SEMÁNTICO	Tipo incompatible, símbolo inexistente, llamada sin sobrecarga válida, archivo .z con nombre incorrecto.
PROYECTO	Cero o más de un .pig principal, importación no resoluble.

La compilación aplica un gate global: todos los archivos deben ser válidos y la semántica completa debe terminar sin errores. La interfaz limpia resultados anteriores y se detiene inmediatamente si el proyecto no es válido.

Tabla de compatibilidad de tipos para los tres lenguajes

Las tablas siguientes documentan el comportamiento implementado. Las promociones solo se aplican a tipos primitivos numéricos; arreglos, estructuras y objetos requieren compatibilidad de referencia/tipo exacta.

Y?

Tipos primitivos: caracter < entero < flotante para promoción numérica. bool no es numérico. cadena se concatena únicamente con +.

Izq. \ Der.	caracter	entero	flotante	bool	cadena
caracter	caracter	entero	flotante	Error	cadena*
entero	entero	entero	flotante	Error	cadena*
flotante	flotante	flotante	flotante	Error	cadena*
bool	Error	Error	Error	bool solo lógica	cadena*
cadena	cadena*	cadena*	cadena*	cadena*	cadena*

* En suma (+) el resultado es cadena. En -, * y / una cadena produce error.

Destino	Origen permitido
caracter	caracter
entero	caracter, entero
flotante	caracter, entero, flotante
bool	bool
cadena	cadena
arreglo	mismo tipo de arreglo
estructura	misma estructura declarada

Zetariano

Promociones de asignación/sobrecarga: char -> int -> double. null es compatible con referencias. String concatena con tipos primitivos usando +.

Izq. \ Der.	char	int	double	boolean	String
char	int	int	double	Error	String*
int	int	int	double	Error	String*
double	double	double	double	Error	String*
boolean	Error	Error	Error	boolean solo lógica	String*
String	String*	String*	String*	String*	String*

* En suma (+) el resultado es String. Objetos y arreglos no se concatenan directamente.

Destino	Origen permitido
char	char
int	char, int
double	char, int, double
boolean	boolean
String	String, null
objeto	misma clase, null
arreglo	mismo tipo/dimensiones, null según referencia

Pig Latin

Promociones: littera -> numerus -> decimalis. bool es lógico. textum concatena con tipos primitivos mediante +.

Izq. \ Der.	littera	numerus	decimalis	bool	textum
littera	numerus	numerus	decimalis	Error	textum*
numerus	numerus	numerus	decimalis	Error	textum*
decimalis	decimalis	decimalis	decimalis	Error	textum*
bool	Error	Error	Error	bool solo lógica	textum*
textum	textum*	textum*	textum*	textum*	textum*

* En suma (+) el resultado es textum. Estructuras, objetos y arreglos no se concatenan directamente.

Destino	Origen permitido
littera	littera
numerus	littera, numerus
decimalis	littera, numerus, decimalis
bool	bool
textum	textum
estructura importada	misma estructura
objeto Z importado	misma clase/referencia compatible
arreglo	mismo tipo de elemento y dimensiones compatibles

Operadores lógicos y relacionales

En los tres lenguajes, && y || requieren booleanos y producen booleano. La negación ! también requiere booleano. Las comparaciones <, >, <= y >= se aplican a valores numéricos compatibles. == y != permiten tipos iguales y comparaciones numéricas compatibles; Zetariano además permite comparar null con referencias.

Pruebas y validación

El proyecto se validó con pruebas unitarias y pruebas integrales que recorren todo el pipeline hasta GCC. La suite automatizada supera las 140 pruebas y se agregaron regresiones específicas para errores encontrados durante la auditoría.

Área	Validación
Estructuras Y?	Estructuras anidadas, campos y arreglo dentro de estructura.
Arreglos	Reserva sin inicializador, matrices, defaults y acceso.
Referencias	null=-1, primitivos=0 y arreglos de referencias.
Entrada	leer/readln y lectura Pig tipada, incluida lectura descartada.
Sobrecarga Z	Constructores y métodos int/double, promociones char->int e int->double.
Cadenas	Parámetros, atributos, String[], textum[] y concatenación.
División/salida Z	División entera, /=, char y boolean.
Controles	if/switch/for/while/do-while, break, continue, ternario y recursividad.
Gate de errores	Un archivo roto bloquea cuartetos, memoria, C3D y C.
Nombre clase Z	Se obtiene del AST y no de comentarios.
Pig	##...##, sintaxis << y exactamente un .pig.
Temporales	Literales t1/L2 e identificadores t0/L0 sin colisión.
Paso por referencia Y?	Modificación de arreglo visible en el llamador.

Comandos de verificación

mvn clean test

mvn exec:java

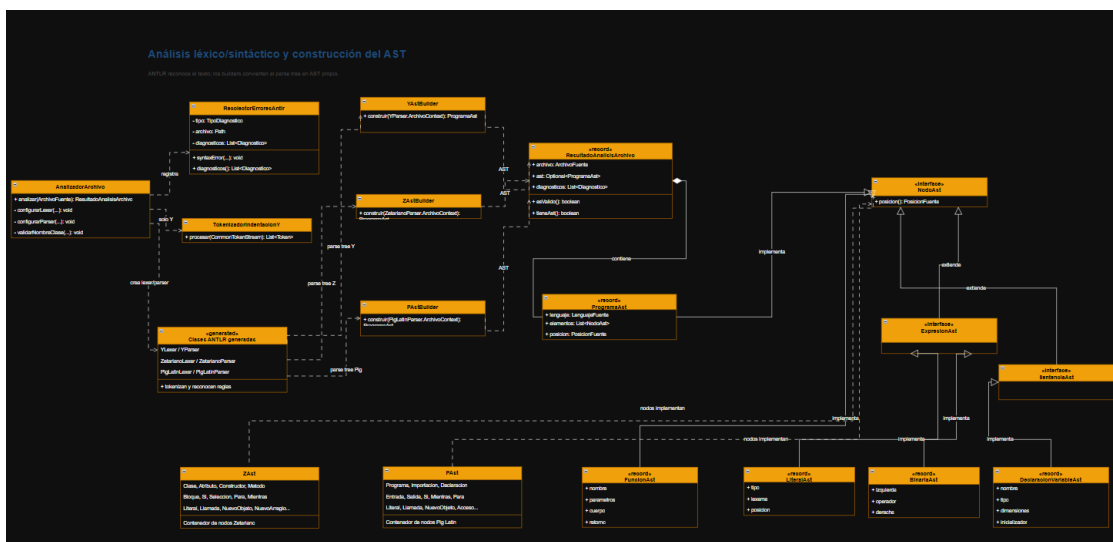
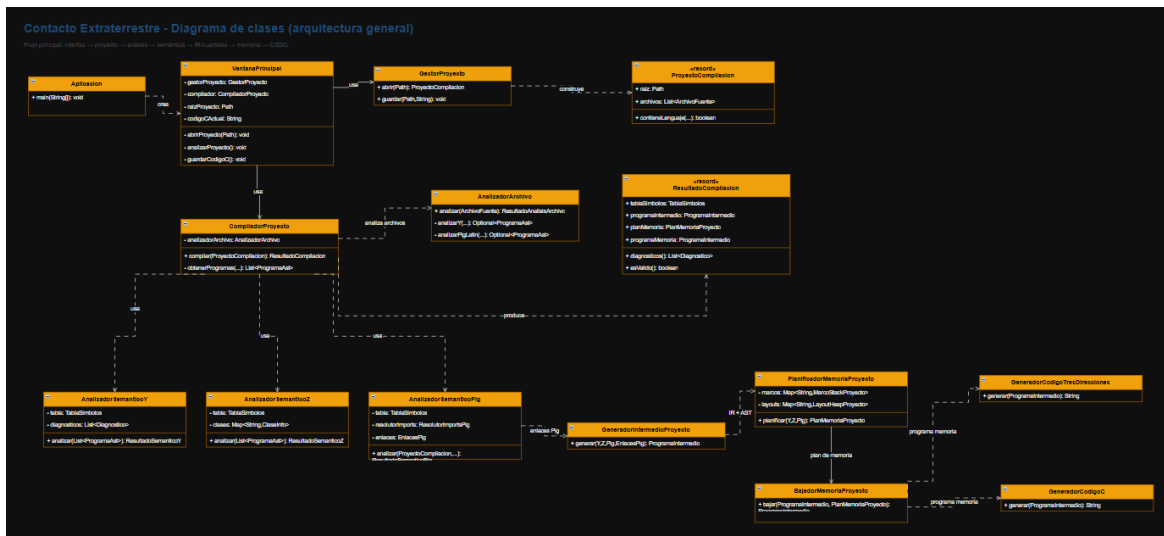
Después de exportar el C:

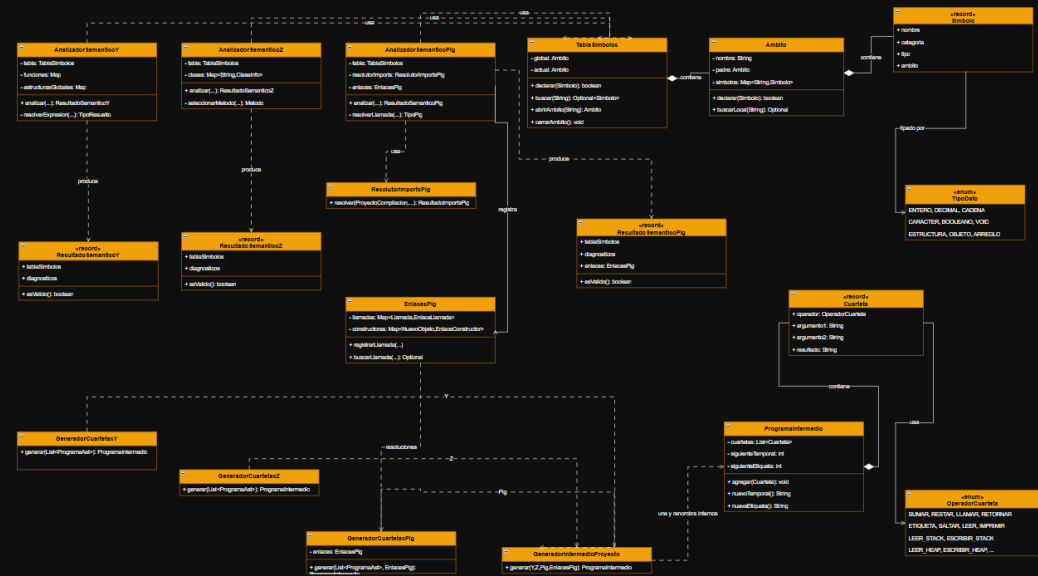
```
gcc programa.c -o programa.exe -lm
./programa.exe
```

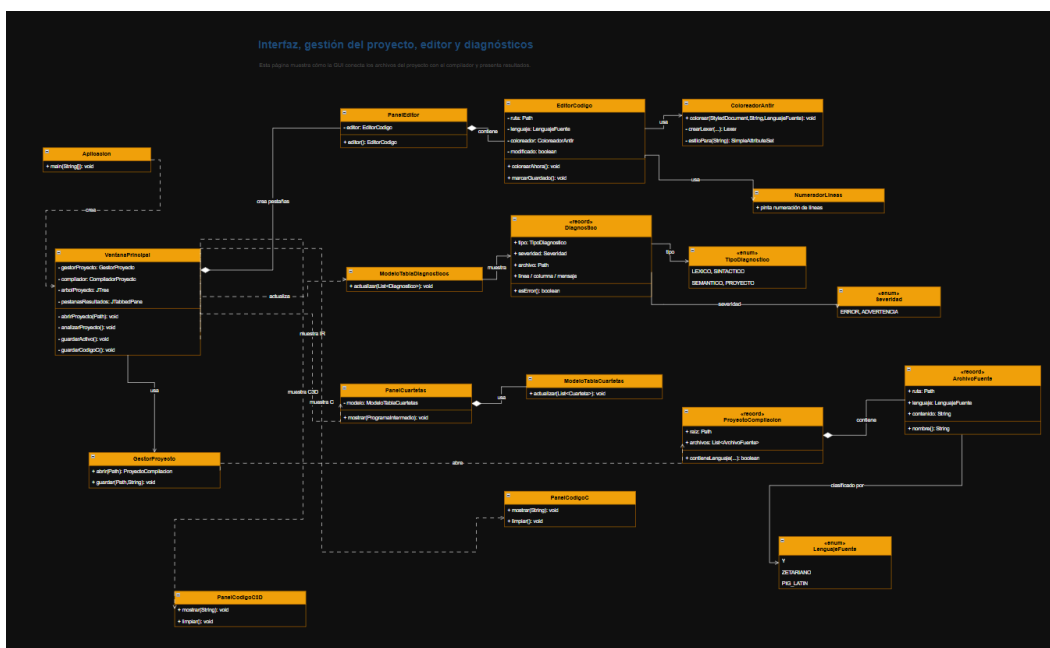
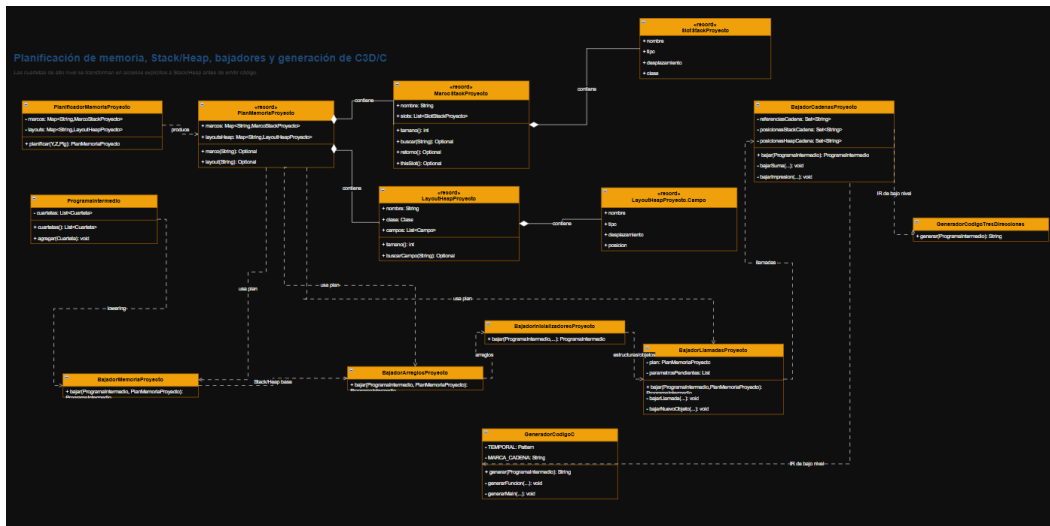
Clases más importantes y complejas

Clase	Qué hace	Por qué es importante
CompiladorProyecto	Orquesta todo el proceso y decide si se puede generar backend.	Es la fachada del compilador.
AnalizadorArchivo	Selecciona lexer/parser, recolecta errores y construye el AST correspondiente.	Une ANTLR con el modelo propio.
AnalizadorSemanticoY/Z/Pig	Resuelven tipos, símbolos, llamadas, contexto y compatibilidad.	Son la principal barrera antes de generar.
ResolutorImportsPig	Une el programa principal con funciones/estructuras/clases externas.	Permite integración multilenguaje.
GeneradorCuartetas*	Transforman AST a operaciones intermedias comunes.	Separan el lenguaje fuente del backend.
GeneradorIntermedioProyecto	Une programas y protege nombres de temporales/etiquetas.	Evita colisiones entre lenguajes y usuario.
PlanificadorMemoriaProyecto	Calcula offsets de Stack y layouts de Heap.	Define dónde vive cada dato.
BajadorLlamadasProyecto	Implementa marcos, parámetros, this/receptores, constructores y retorno.	Clave para métodos, sobrecarga y recursividad.
BajadorArreglosProyecto	Linealiza índices y genera accesos Heap.	Clave para matrices y referencias.
BajadorCadenasProyecto	Gestiona referencias de String y concatenaciones.	Las cadenas no caben directamente en un double.
GeneradorCodigoC	Traduce el IR de memoria a C compilable.	Última fase del backend.
VentanaPrincipal	Coordina árbol, editores, análisis y pestañas.	Conecta toda la lógica con el usuario.

Diagrama de clases







<https://github.com/MaYan711/Proyecto-1-Contacto-3xtrat3rr3str3D>